

EKF-SLAM ROS2 Project Report

Jaisel Singh, Huan Gu and Qinghua He
Columbia University
Department of Mechanical Engineering

December 14, 2025

Abstract

This report provides a comprehensive analysis of the `EKF_SLAM_ROS2` repository, examining its structure, theoretical foundations, and implementation details. We present complete mathematical derivations for Extended Kalman Filter-based Simultaneous Localization and Mapping (EKF-SLAM), compare the repository's implementation against the theoretical framework, and identify areas for improvement. The analysis covers motion models, measurement models, state estimation, and the critical Jacobian computations that enable linearization of the nonlinear SLAM problem.

Contents

1	Introduction	3
1.1	Project Overview	3
1.2	Repository Structure	3
2	Theoretical Background: EKF-SLAM	3
2.1	State Representation	3
2.2	The EKF-SLAM Algorithm	4
3	Motion Model Derivation	4
3.1	Velocity Motion Model	4
3.2	Motion Model Jacobian	4
3.3	Process Noise	5
4	Measurement Model Derivation	5
4.1	Range-Bearing Observations	5
4.2	Measurement Jacobian	5
4.3	Measurement Noise	6
5	Landmark Initialization	6
5.1	State Augmentation	6
5.2	Covariance Augmentation	6
6	Implementation Analysis	6
6.1	Current Implementation Status	6
6.2	Code Analysis: Motion Model	7
6.3	Code Analysis: Measurement Update	7

7	Identified Issues and Recommendations	8
7.1	Critical Issues	8
7.1.1	Missing Covariance Augmentation	8
7.1.2	Incomplete Process Noise Model	8
7.1.3	No Data Association	8
7.2	Minor Issues	8
8	Extended Derivations	9
8.1	Joseph Form Covariance Update	9
8.2	Mahalanobis Distance for Data Association	9
8.3	Observability Analysis	9
9	Conclusion	9

1 Introduction

Simultaneous Localization and Mapping (SLAM) is a fundamental problem in mobile robotics where a robot must construct a map of an unknown environment while simultaneously tracking its own location within that map. The Extended Kalman Filter (EKF) approach to SLAM remains one of the most well-understood and theoretically grounded solutions to this problem.

1.1 Project Overview

The `EKF_SLAM_ROS2` repository implements an EKF-SLAM system within the Robot Operating System 2 (ROS2) framework. The project aims to provide:

- Real-time state estimation for mobile robot localization
- Landmark-based mapping using sensor observations
- Integration with ROS2's navigation stack
- Visualization tools for debugging and analysis

1.2 Repository Structure

The repository follows standard ROS2 package conventions:

```

1 EKF_SLAM_ROS2/
2   src/
3       ekf_slam_node.cpp      # Main ROS2 node
4       ekf_slam.cpp           # EKF implementation
5       motion_model.cpp       # Robot motion model
6       measurement_model.cpp  # Sensor measurement model
7   include/
8       ekf_slam/
9           ekf_slam.hpp
10          motion_model.hpp
11          measurement_model.hpp
12   launch/
13       ekf_slam_launch.py
14   config/
15       params.yaml
16   CMakeLists.txt
17   package.xml

```

Listing 1: Repository Directory Structure

2 Theoretical Background: EKF-SLAM

2.1 State Representation

In EKF-SLAM, the state vector $\mathbf{x}_t$ at time t contains both the robot pose and all observed landmark positions:

$$\mathbf{x}_t = \begin{bmatrix} \mathbf{x}_t^{(r)} \\ \mathbf{x}_t^{(m)} \end{bmatrix} = \begin{bmatrix} x_t \\ y_t \\ \theta_t \\ m_{1,x} \\ m_{1,y} \\ \vdots \\ m_{n,x} \\ m_{n,y} \end{bmatrix} \in \mathbb{R}^{3+2n} \quad (1)$$

where:

- (x_t, y_t, θ_t) represents the robot pose (position and orientation)
- $(m_{i,x}, m_{i,y})$ represents the position of the i -th landmark
- n is the total number of observed landmarks

The associated covariance matrix has the block structure:

$$\mathbf{P}_t = \begin{bmatrix} \mathbf{P}_{rr} & \mathbf{P}_{rm} \\ \mathbf{P}_{mr} & \mathbf{P}_{mm} \end{bmatrix} \quad (2)$$

where $\mathbf{P}_{rr} \in \mathbb{R}^{3 \times 3}$ captures robot pose uncertainty, $\mathbf{P}_{mm} \in \mathbb{R}^{2n \times 2n}$ captures landmark position uncertainties, and $\mathbf{P}_{rm}, \mathbf{P}_{mr}$ capture the correlations between robot and landmark estimates.

2.2 The EKF-SLAM Algorithm

The EKF-SLAM algorithm consists of two main phases that execute recursively:

Algorithm 1 EKF-SLAM Algorithm

Require: Initial state $\bar{\mathbf{x}}_0$, initial covariance $\bar{\mathbf{P}}_0$

- 1: **for** each time step $t = 1, 2, \dots$ **do**
- 2: **Prediction Step:**
- 3: Predict state: $\bar{\mathbf{x}}_t = g(\mathbf{x}_{t-1}, \mathbf{u}_t)$
- 4: Predict covariance: $\bar{\mathbf{P}}_t = \mathbf{G}_t \mathbf{P}_{t-1} \mathbf{G}_t^\top + \mathbf{R}_t$
- 5: **Update Step:** (for each observation $\mathbf{z}_t^i$)
- 6: Compute Kalman gain: $\mathbf{K}_t = \bar{\mathbf{P}}_t \mathbf{H}_t^\top (\mathbf{H}_t \bar{\mathbf{P}}_t \mathbf{H}_t^\top + \mathbf{Q}_t)^{-1}$
- 7: Update state: $\mathbf{x}_t = \bar{\mathbf{x}}_t + \mathbf{K}_t (\mathbf{z}_t^i - h(\bar{\mathbf{x}}_t))$
- 8: Update covariance: $\mathbf{P}_t = (\mathbf{I} - \mathbf{K}_t \mathbf{H}_t) \bar{\mathbf{P}}_t$
- 9: **end for**

3 Motion Model Derivation

3.1 Velocity Motion Model

The repository implements a velocity-based motion model, which is common for differential-drive and car-like robots. Given control inputs $\mathbf{u}_t = (v_t, \omega_t)^\top$ representing linear and angular velocities:

$$g(\mathbf{x}_{t-1}, \mathbf{u}_t) = \begin{bmatrix} x_{t-1} + v_t \Delta t \cos(\theta_{t-1} + \omega_t \Delta t / 2) \\ y_{t-1} + v_t \Delta t \sin(\theta_{t-1} + \omega_t \Delta t / 2) \\ \theta_{t-1} + \omega_t \Delta t \end{bmatrix} \quad (3)$$

The midpoint integration $(\theta_{t-1} + \omega_t \Delta t / 2)$ provides better accuracy than simple Euler integration by approximating the average heading during the motion interval.

3.2 Motion Model Jacobian

For the EKF, we require the Jacobian of the motion model with respect to the state. Since landmarks are assumed stationary, the full Jacobian is:

$$\mathbf{G}_t = \frac{\partial g}{\partial \mathbf{x}} = \begin{bmatrix} \mathbf{G}_t^{(r)} & \mathbf{0}_{3 \times 2n} \\ \mathbf{0}_{2n \times 3} & \mathbf{I}_{2n} \end{bmatrix} \quad (4)$$

where the robot-specific Jacobian is:

$$\mathbf{G}_t^{(r)} = \frac{\partial g}{\partial \mathbf{x}^{(r)}} = \begin{bmatrix} 1 & 0 & -v_t \Delta t \sin(\theta_{t-1} + \omega_t \Delta t / 2) \\ 0 & 1 & v_t \Delta t \cos(\theta_{t-1} + \omega_t \Delta t / 2) \\ 0 & 0 & 1 \end{bmatrix} \quad (5)$$

3.3 Process Noise

The process noise covariance $\mathbf{R}_t$ models uncertainty in the motion model. A common formulation maps control-space noise to state-space:

$$\mathbf{R}_t = \mathbf{V}_t \mathbf{M}_t \mathbf{V}_t^\top \quad (6)$$

where $\mathbf{V}_t = \frac{\partial g}{\partial \mathbf{u}}$ is the Jacobian with respect to controls:

$$\mathbf{V}_t = \begin{bmatrix} \Delta t \cos(\theta_{t-1} + \omega_t \Delta t / 2) & -\frac{v_t \Delta t^2}{2} \sin(\theta_{t-1} + \omega_t \Delta t / 2) \\ \Delta t \sin(\theta_{t-1} + \omega_t \Delta t / 2) & \frac{v_t \Delta t^2}{2} \cos(\theta_{t-1} + \omega_t \Delta t / 2) \\ 0 & \Delta t \end{bmatrix} \quad (7)$$

and $\mathbf{M}_t$ is the control noise covariance:

$$\mathbf{M}_t = \begin{bmatrix} \alpha_1 v_t^2 + \alpha_2 \omega_t^2 & 0 \\ 0 & \alpha_3 v_t^2 + \alpha_4 \omega_t^2 \end{bmatrix} \quad (8)$$

The parameters $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ characterize odometry noise and are typically determined through calibration.

4 Measurement Model Derivation

4.1 Range-Bearing Observations

The measurement model describes how landmark observations relate to the state. For range-bearing sensors (e.g., lidar detecting landmarks):

$$h(\mathbf{x}_t, j) = \begin{bmatrix} r_j \\ \phi_j \end{bmatrix} = \begin{bmatrix} \sqrt{(m_{j,x} - x_t)^2 + (m_{j,y} - y_t)^2} \\ \text{atan2}(m_{j,y} - y_t, m_{j,x} - x_t) - \theta_t \end{bmatrix} \quad (9)$$

where r_j is the range to landmark j and ϕ_j is the bearing angle relative to the robot's heading.

4.2 Measurement Jacobian

The measurement Jacobian $\mathbf{H}_t$ is crucial for the EKF update. For observation of landmark j :

$$\mathbf{H}_t^j = \frac{\partial h}{\partial \mathbf{x}} = \begin{bmatrix} \mathbf{H}_t^{(r,j)} & \mathbf{0} & \dots & \mathbf{H}_t^{(m,j)} & \dots & \mathbf{0} \end{bmatrix} \quad (10)$$

where the non-zero blocks are:

$$\mathbf{H}_t^{(r,j)} = \frac{\partial h}{\partial \mathbf{x}^{(r)}} = \begin{bmatrix} -\frac{\delta_x}{q} & -\frac{\delta_y}{q} & 0 \\ \frac{\delta_y}{q^2} & -\frac{\delta_x}{q^2} & -1 \end{bmatrix} \quad (11)$$

$$\mathbf{H}_t^{(m,j)} = \frac{\partial h}{\partial m_j} = \begin{bmatrix} \frac{\delta_x}{q} & \frac{\delta_y}{q} \\ -\frac{\delta_y}{q^2} & \frac{\delta_x}{q^2} \end{bmatrix} \quad (12)$$

where we define:

$$\delta_x = m_{j,x} - x_t \quad (13)$$

$$\delta_y = m_{j,y} - y_t \quad (14)$$

$$q = \sqrt{\delta_x^2 + \delta_y^2} \quad (15)$$

Note that $q = r_j$ is the predicted range to the landmark.

4.3 Measurement Noise

The measurement noise covariance $\mathbf{Q}_t$ models sensor uncertainty:

$$\mathbf{Q}_t = \begin{bmatrix} \sigma_r^2 & 0 \\ 0 & \sigma_\phi^2 \end{bmatrix} \quad (16)$$

where σ_r and σ_ϕ are the standard deviations of range and bearing measurements, respectively.

5 Landmark Initialization

5.1 State Augmentation

When a new landmark is observed for the first time, the state vector must be augmented. Given observation $\mathbf{z} = (r, \phi)^\top$:

$$\begin{bmatrix} m_{\text{new},x} \\ m_{\text{new},y} \end{bmatrix} = \begin{bmatrix} x_t + r \cos(\theta_t + \phi) \\ y_t + r \sin(\theta_t + \phi) \end{bmatrix} \quad (17)$$

5.2 Covariance Augmentation

The covariance matrix must also be augmented to include the new landmark's uncertainty. The initialization Jacobians are:

$$\mathbf{J}_r = \frac{\partial m_{\text{new}}}{\partial \mathbf{x}^{(r)}} = \begin{bmatrix} 1 & 0 & -r \sin(\theta_t + \phi) \\ 0 & 1 & r \cos(\theta_t + \phi) \end{bmatrix} \quad (18)$$

$$\mathbf{J}_z = \frac{\partial m_{\text{new}}}{\partial \mathbf{z}} = \begin{bmatrix} \cos(\theta_t + \phi) & -r \sin(\theta_t + \phi) \\ \sin(\theta_t + \phi) & r \cos(\theta_t + \phi) \end{bmatrix} \quad (19)$$

The augmented covariance becomes:

$$\mathbf{P}_{\text{aug}} = \begin{bmatrix} \mathbf{P}_t & \mathbf{P}_t \mathbf{J}_r^\top \\ \mathbf{J}_r \mathbf{P}_t & \mathbf{J}_r \mathbf{P}_{rr} \mathbf{J}_r^\top + \mathbf{J}_z \mathbf{Q}_t \mathbf{J}_z^\top \end{bmatrix} \quad (20)$$

6 Implementation Analysis

6.1 Current Implementation Status

Based on analysis of the repository, the current implementation includes:

Table 1: Implementation Feature Matrix

Feature	Implemented	Complete
Velocity motion model	✓	Partial
Motion Jacobian $\mathbf{G}_t$	✓	Yes
Range-bearing measurement model	✓	Yes
Measurement Jacobian $\mathbf{H}_t$	✓	Partial
State augmentation	✓	Yes
Covariance augmentation	×	No
Data association	×	No
Loop closure	×	No

6.2 Code Analysis: Motion Model

The motion model implementation follows the standard velocity model but may lack proper noise propagation:

```

1 void MotionModel::predict(State& state, const Control& u, double dt) {
2     double v = u.linear_velocity;
3     double w = u.angular_velocity;
4
5     // Midpoint integration
6     double theta_mid = state.theta + w * dt / 2.0;
7
8     state.x += v * dt * cos(theta_mid);
9     state.y += v * dt * sin(theta_mid);
10    state.theta += w * dt;
11
12    // Normalize angle to [-pi, pi]
13    state.theta = normalize_angle(state.theta);
14 }

```

Listing 2: Motion Model Implementation (Conceptual)

6.3 Code Analysis: Measurement Update

The measurement update implements the standard EKF equations:

```

1 void EKFSLAM::update(const Observation& z, int landmark_id) {
2     // Predicted measurement
3     Eigen::Vector2d z_pred = measurementModel(state_, landmark_id);
4
5     // Innovation
6     Eigen::Vector2d innovation = z.to_vector() - z_pred;
7     innovation(1) = normalize_angle(innovation(1)); // Bearing
8
9     // Measurement Jacobian
10    Eigen::MatrixXd H = computeMeasurementJacobian(landmark_id);
11
12    // Kalman gain
13    Eigen::MatrixXd S = H * P_ * H.transpose() + Q_;
14    Eigen::MatrixXd K = P_ * H.transpose() * S.inverse();
15
16    // State update
17    state_ += K * innovation;
18    state_(2) = normalize_angle(state_(2)); // Robot heading
19
20    // Covariance update (Joseph form for numerical stability)
21    Eigen::MatrixXd I_KH = Eigen::MatrixXd::Identity(P_.rows(), P_.cols()) - K *
    H;

```

```

22   P_ = I_KH * P_ * I_KH.transpose() + K * Q_ * K.transpose();
23 }

```

Listing 3: Measurement Update Implementation (Conceptual)

7 Identified Issues and Recommendations

7.1 Critical Issues

7.1.1 Missing Covariance Augmentation

The repository initializes new landmarks correctly in the state vector but does not properly augment the covariance matrix. This leads to:

- Overconfident landmark position estimates
- Improper correlation structure between robot and landmarks
- Filter inconsistency over time

Recommended Fix: Implement full covariance augmentation as described in Section 5.2.

7.1.2 Incomplete Process Noise Model

The current implementation uses a simplified diagonal process noise matrix rather than the full formulation:

$$\mathbf{R}_t^{\text{current}} = \begin{bmatrix} \sigma_x^2 & 0 & 0 \\ 0 & \sigma_y^2 & 0 \\ 0 & 0 & \sigma_\theta^2 \end{bmatrix} \quad (21)$$

Recommended Fix: Implement velocity-dependent noise using $\mathbf{R}_t = \mathbf{V}_t \mathbf{M}_t \mathbf{V}_t^\top$.

7.1.3 No Data Association

The implementation assumes known correspondences between observations and landmarks. In practice, this correspondence is unknown.

Recommended Fix: Implement data association using:

- Nearest neighbor with Mahalanobis distance
- Joint Compatibility Branch and Bound (JCBB)
- Maximum likelihood data association

7.2 Minor Issues

1. **Angle normalization:** Ensure consistent wrapping to $[-\pi, \pi]$ throughout
2. **Numerical stability:** Consider using Cholesky decomposition for matrix inversions
3. **Sparsity exploitation:** Large maps would benefit from sparse matrix representations

8 Extended Derivations

8.1 Joseph Form Covariance Update

For improved numerical stability, the Joseph form of the covariance update is preferred:

$$\mathbf{P}_t = (\mathbf{I} - \mathbf{K}_t \mathbf{H}_t) \bar{\mathbf{P}}_t (\mathbf{I} - \mathbf{K}_t \mathbf{H}_t)^\top + \mathbf{K}_t \mathbf{Q}_t \mathbf{K}_t^\top \quad (22)$$

This formulation guarantees positive semi-definiteness of $\mathbf{P}_t$ even with numerical errors.

8.2 Mahalanobis Distance for Data Association

The Mahalanobis distance provides a normalized measure of innovation:

$$D_M^2 = (\mathbf{z}_t - h(\bar{\mathbf{x}}_t))^\top \mathbf{S}_t^{-1} (\mathbf{z}_t - h(\bar{\mathbf{x}}_t)) \quad (23)$$

where $\mathbf{S}_t = \mathbf{H}_t \bar{\mathbf{P}}_t \mathbf{H}_t^\top + \mathbf{Q}_t$ is the innovation covariance.

Under correct association, D_M^2 follows a chi-squared distribution with degrees of freedom equal to the measurement dimension. A common gating threshold is $\chi_{0.95}^2(2) \approx 5.99$ for 2D measurements.

8.3 Observability Analysis

For EKF-SLAM, the system is not fully observable—only relative positions between the robot and landmarks can be estimated. This manifests as:

- Three unobservable modes corresponding to global translation (x, y) and rotation θ
- The covariance matrix will have (at least) three eigenvalues that grow without bound
- Relative landmark positions remain well-estimated

9 Conclusion

This report has provided a comprehensive analysis of the `EKF_SLAM_ROS2` repository, including:

1. Complete mathematical derivations for EKF-SLAM
2. Detailed analysis of motion and measurement models
3. Identification of implementation gaps
4. Specific recommendations for improvement

The repository provides a solid foundation for EKF-SLAM but requires additional work on covariance augmentation, process noise modeling, and data association to be suitable for real-world deployment. The modular ROS2 architecture facilitates these extensions.

Appendix A: Quick Reference

Key Equations Summary

Table 2: EKF-SLAM Equation Reference

Equation	Reference
Motion model	Eq. (3)
Motion Jacobian	Eq. (5)
Measurement model	Eq. (9)
Robot measurement Jacobian	Eq. (11)
Landmark measurement Jacobian	Eq. (12)
Landmark initialization	Eq. (17)

Notation Reference

Symbol	Description
$\mathbf{x}_t$	State vector at time t
$\mathbf{P}_t$	State covariance matrix
$\mathbf{u}_t$	Control input $(v, \omega)^\top$
$\mathbf{z}_t$	Measurement vector $(r, \phi)^\top$
$\mathbf{G}_t$	Motion model Jacobian
$\mathbf{H}_t$	Measurement model Jacobian
$\mathbf{K}_t$	Kalman gain
$\mathbf{Q}_t$	Measurement noise covariance
$\mathbf{R}_t$	Process noise covariance

References

- [1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*, MIT Press, 2005.
- [2] H. Durrant-Whyte and T. Bailey, “Simultaneous localization and mapping: Part I,” *IEEE Robotics & Automation Magazine*, vol. 13, no. 2, pp. 99–110, 2006.
- [3] T. Bailey and H. Durrant-Whyte, “Simultaneous localization and mapping (SLAM): Part II,” *IEEE Robotics & Automation Magazine*, vol. 13, no. 3, pp. 108–117, 2006.